

Relatório de Auditoria de Segurança — SecurePlay

Revisão integral do estado atual do código, com foco em isolamento de tenant/dono, autorização server-side, IDOR, segredos e XSS.

DATA	ESCOPO
30 de agosto de 2026	Backend, frontend, auth, MySQL, Redis, Socket.IO, S3, Git e bundle
ESTADO AUDITADO	NOTA METODOLÓGICA
Worktree local atual, incluindo alterações não commitadas	57 handlers HTTP, gateway, sinks XSS, 102 commits e 114 blobs Git inalcançáveis

Mapeamento por stack: MySQL/TypeORM usa filtros manuais por usuario_id/empresa_id; NestJS usa JWT e RolesGuard globais; React/Vite foi revisado por gates, sinks e bundle; Git foi examinado em todas as refs e objetos alcançáveis/inalcançáveis.

01 Resumo executivo

Foram confirmados 9 achados: 7 de severidade alta e 2 de severidade média. Os riscos centrais são confiança em identidade fornecida pelo cliente, controles de elegibilidade mantidos apenas no frontend, ausência de tenant-awareness em pontos administrativos, tooling de desenvolvimento que grava código e credenciais reutilizáveis.



Limites: análise estática e build local. Não houve conexão com MySQL/Redis/S3, teste de credenciais externas, DAST em deployment real ou publicação de dados. O segredo histórico foi redigido. Nenhuma issue foi criada no GitHub.

03 Pontos fortes e pontos fracos

PONTOS FORTES	PONTOS FRACOS
RBAC server-side JWT e RolesGuard são globais; os 20 handlers privilegiados Nest possuem @Roles(ADMIN).	Identidade do socket Room deriva de query.userId, não do token.
Admin por empresa Tema/logo/usuários/convites derivam a empresa do caller; revogação usa {id, empresa_id}.	Tenant admin OwnershipGuard libera ADMIN e papéis de empresa/plataforma são indistintos.
Convites Token aleatório de 32 bytes, SHA-256 em repouso, validade/uso, lock pessimista e empresa herdada.	Regras só na UI Aulas locked e daily challenge não são impostos em toda rota sensível.
Owner por usuário Achievements, arcade, progresso de conteúdo, stats e duas ações unitárias de notificação usam caller/comparação composta.	Tooling exposto Vite em host:true grava TypeScript e aceita valores sem validação runtime.
XSS comum React mantém escaping; não há raw HTML, Markdown, templates HTML ou e-mail; avatar codifica o nome.	Credenciais Seed pública no startup e segredo JWT preservado em ref histórica.
Segredos atuais JWT ausente impede startup; .env está ignorado e seus valores não aparecem no Git nem no bundle atual.	

Cobertura de controllers e handlers

Arquivo	Handlers	Resultado	Status
app.controller.ts	2	JWT global; sem acesso a objeto de terceiro	Correto
auth.controller.ts	4 / 5 paths	Login público; me/token/password vinculados ao caller	Correto
usuario.controller.ts	0	Sem handlers	N/A
admin.controller.ts	7	Tema, logo, usuários e convites por empresa do caller	Correto
convites.controller.ts	2	Token 256-bit, hash, validade, lock e empresa herdada	Correto
achievements.controller.ts	5	Wallet, métricas e cosméticos pelo caller	Correto
arcade.controller.ts	4	Run Redis usa caller + UUID; stats pelo caller	Correto
benefits.controller.ts	3	GET público; mutações globais pelo ADMIN de empresa	SEC-05
challenge.controller.ts	5	Owner correto; elegibilidade daily ausente em 3	SEC-04
modulo.controller.ts	5	Progresso do caller; CRUD global por ADMIN	SEC-05
aula.controller.ts	7	3 escritas owner-safe; GET locked e CRUD global	SEC-03/05
aula-quiz.controller.ts	3	CRUD global por ADMIN	SEC-05
upload.controller.ts	1	Presign global sem empresa/lookup	SEC-05
dashboard.controller.ts	4	Caller em stats/streak; scope company filtra empresa	Correto
notification.controller.ts	5	2 ações owner-safe; 3 bypassam tenant para ADMIN	SEC-02

Arquivo	Handlers	Resultado	Status
Socket.IO gateway	1 conexão	Identidade declarada na query	SEC-01

04 Achados detalhados por categoria

Severidade	Arquivo:linha	Descrição
ALTA	backend/src/gateway/app.gateway.ts:24-35 backend/src/gateway/app.gateway.ts:45-56 frontend/src/hooks/useSocket.ts:12-14	SEC-01 - WebSocket aceita identidade declarada pelo cliente O handshake converte query.userId em identidade e entra na sala user_<id> sem validar JWT, cookie, expiração ou blacklist. O fanout envia título, mensagem, tipo e data para essa sala. Exploração: Um cliente Socket.IO escolhe o ID numérico de outra pessoa e aguarda uma nova notificação. CORS não autentica clientes nativos e não vincula a sala ao sujeito do token.
ALTA	backend/src/auth/ownership.guard.ts:23-25 backend/src/notification/notification.controller.ts:26-48 backend/src/notification/notification.controller.ts:70-76 backend/src/notification/notification.service.ts:21-43 backend/src/notification/notification.service.ts:99-135	SEC-02 - Administrador ignora tenant nas rotas de notificação O OwnershipGuard retorna true para qualquer ADMIN antes de ler o campo de dono. As rotas permitem criar, listar e marcar todas as notificações para um usuario_id informado pelo cliente; o service não compara empresa_id. Exploração: Um administrador da empresa A informa o ID de um usuário da empresa B e lê o histórico, insere uma mensagem ou marca todas as notificações da vítima como lidas.
MÉDIA	frontend/src/components/sections/HomePage/Conteudos/AulaListItem.tsx:19-26 backend/src/conteudo/modulo/modulo.service.ts:99-120 backend/src/conteudo/aula/aula.controller.ts:27-29 backend/src/conteudo/aula/aula.service.ts:49-100 backend/src/conteudo/aula/aula.service.ts:354-377	SEC-03 - Aula bloqueada pode ser lida diretamente por ID O frontend desabilita a aula com status locked, mas GET /conteudo/aulas/:id não chama a verificação de pré-requisito. A resposta inclui vídeo, páginas com URL S3 assinada e perguntas do quiz. Exploração: Copiar um ID locked retornado pelo módulo e chamar diretamente o GET da aula.
MÉDIA	README.md:24,38 backend/src/challenge/challenge.service.ts:36-58 backend/src/challenge/challenge.service.ts:73-87 backend/src/challenge/challenge.controller.ts:29-56 backend/src/challenge/challenge.service.ts:113-173 backend/src/challenge/challenge.service.ts:175-230 backend/src/challenge/challenge.service.ts:265-295	SEC-04 - Backend não impõe o desafio diário selecionado Um desafio é escolhido e cacheado por usuário até o fim do dia, porém perguntas, progresso e submit aceitam qualquer challengeld ativo e nunca o comparam ao valor daily. Exploração: Enumerar IDs ativos, usar /progress (que revela correct) e submeter vários desafios para acumular XP no mesmo dia.
ALTA	backend/src/auth/roles.enum.ts:1-4 backend/src/admin/admin.controller.ts:11-12 backend/src/conteudo/modulo/modulo.controller.ts:31-49 backend/src/conteudo/aula/aula.controller.ts:32-50 backend/src/conteudo/aula-quiz/aula-quiz.controller.ts:24-45 backend/src/conteudo/s3/upload.controller.ts:31-44 backend/src/benefits/benefits.controller.ts:21-24,35-38 backend/src/conteudo/modulo/modulo.entity.ts:20-60 backend/src/conteudo/aula/aula.entity.ts:15-55 backend/src/conteudo/s3/s3.service.ts:59-72	SEC-05 - Administrador de empresa controla catálogos e mídia globais O único papel ADMIN também administra uma empresa, mas esse mesmo papel altera objetos globais sem requester ou empresa_id: módulos, aulas, quizzes, Benefits e chaves de mídia. Exploração: Um administrador da empresa A chama os CRUDs por ID e altera, exclui ou sobrescreve treinamento consumido por todas as empresas.

Severidade	Arquivo:linha	Descrição
ALTA	frontend/vite.config.ts:5-7,13-15 frontend/src/app/App.tsx:25-27 frontend/src/prototypes/worldmap/WorldMapPage.tsx:64-68,99-104 frontend/src/prototypes/worldmap/devSaveRegionPlugin.ts:10-12,19-39	SEC-06 - Endpoint do editor Vite sobrescreve fonte sem autenticação O Vite registra um middleware de escrita, escuta em todas as interfaces e expõe a rota do editor sem sessão ou papel. POST /__save-region lê e sobrescreve mockData.ts. Exploração: Qualquer cliente que alcance a porta 5173 envia um biomeld e pontos e altera o repositório.
ALTA	frontend/src/prototypes/worldmap/devSaveRegionPlugin.ts:19-31 frontend/src/prototypes/worldmap/devSaveRegionPlugin.ts:87-94 frontend/src/prototypes/worldmap/WorldMapPage.tsx:9,16	SEC-07 - Coordenadas viram JavaScript persistente em mockData.ts O cast TypeScript não valida JSON em runtime; apenas Array.isArray é exigido. p.x e p.y são interpolados sem escape em código TypeScript gravado no repositório. Exploração: Uma string em y fecha o objeto, injeta um IIFE e reabre o objeto. Ao importar ou atualizar via HMR, o código roda no origin do frontend e pode agir com a sessão da vítima.
ALTA	backend/src/app.module.ts:12,52 backend/src/seed/seed.service.ts:22-24 backend/src/seed/seed.service.ts:26-40 backend/src/seed/seed.service.ts:42-58 backend/src/auth/auth.controller.ts:26-34 backend/src/auth/auth.service.ts:31-60	SEC-08 - Startup cria dez contas com a senha pública seed123 SeedModule é carregado sem condição e OnModuleInit cria dez e-mails previsíveis com a mesma senha hardcoded. As contas USER sem empresa continuam autenticáveis. Exploração: Entrar no endpoint público de login com um dos e-mails seed e a senha publicada no código.
ALTA	commit 06bbb0c7... / backend/src/auth/constants.ts:3 commit 06bbb0c7... / backend/src/auth/auth.module.ts:7,12-15 commit 3641d626... (remoção, sem limpeza do histórico) refs/remotes/origin/master -> 06bbb0c7...	SEC-09 - Segredo JWT permanece em referência do histórico Git O commit inicial contém um segredo JWT não-placeholder de 32 caracteres e o usa no JwtModule. A remoção posterior não apagou o objeto: origin/master ainda aponta para o commit vulnerável. Exploração: Se algum ambiente ainda aceitar o valor antigo, um atacante que lê o Git pode assinar JWT com sub e role arbitrários, inclusive admin.

05 Evidências, explorabilidade e correção

SEC-01 - WebSocket aceita identidade declarada pelo cliente

ALTA

Categoria: IDOR

Por que é explorável: Um cliente Socket.IO escolhe o ID numérico de outra pessoa e aguarda uma nova notificação. CORS não autentica clientes nativos e não vincula a sala ao sujeito do token.

Impacto: Leitura futura de notificações privadas, inclusive entre empresas.

Condição: Gateway alcançável, ID de vítima válido e emissão de nova notificação.

Correção recomendada: Validar o JWT no handshake, consultar a blacklist e derivar a sala exclusivamente de sub.

Evidência de código:

[backend/src/gateway/app.gateway.ts:24-35](#)

```
const userId = Number(client.handshake.query.userId);
if (!userId || isNaN(userId)) { client.disconnect(); return; }
client.join(`user_${userId}`);
```

[backend/src/gateway/app.gateway.ts:45-56](#)

```
const room = `user_${payload.usuario_id}`;
this.server.to(room).emit('new-notification', payload);
```

SEC-02 - Administrador ignora tenant nas rotas de notificação

ALTA

Categoria: Banco sem tranca

Por que é explorável: Um administrador da empresa A informa o ID de um usuário da empresa B e lê o histórico, insere uma mensagem ou marca todas as notificações da vítima como lidas.

Impacto: Quebra horizontal de confidencialidade e integridade entre tenants.

Condição: Papel ADMIN, múltiplas empresas e ID válido do usuário alvo.

Correção recomendada: Passar o solicitante ao service e exigir igualdade de empresa_id, ou usar guard tenant-aware.

Evidência de código:

[backend/src/auth/ownership.guard.ts:23-25](#)

```
if (user.role === Role.ADMIN) {
  return true;
}
```

[backend/src/notification/notification.service.ts:131-135](#)

```
return this.notificationRepository.find({
  where: { usuario_id },
  order: { created_at: 'DESC' },
});
```

SEC-03 - Aula bloqueada pode ser lida diretamente por ID

MÉDIA

Categoria: Permissão no navegador

Por que é explorável: Copiar um ID locked retornado pelo módulo e chamar diretamente o GET da aula.

Impacto: Acesso antecipado a conteúdo e quiz que a progressão declara bloqueados.

Condição: Usuário autenticado, aula ativa e aula anterior ainda não concluída.

Correção recomendada: Executar verificarDesbloqueio antes de assinar ou devolver qualquer mídia da aula.

Evidência de código:

[frontend/src/components/sections/HomePage/Conteudos/AulaListItem.tsx:19-26](#)

```
const isLocked = aula.status === 'locked';
<button onClick={isLocked ? undefined : onClick} disabled={isLocked}>
```

[backend/src/conteudo/aula/aula.service.ts:49-80](#)

```
const aula = await this.aulaRepository.findOne({
  where: { id, active: true },
});
// sem verificarDesbloqueio
content_url: aula.content_url ? await this.resolveUrl(aula.content_url) : null,
pages: aula.pages ? await Promise.all(aula.pages.map((key) => this.resolveUrl(key))) : null,
```

SEC-04 - Backend não impõe o desafio diário selecionado

MÉDIA

Categoria: Permissão no navegador

Por que é explorável: Enumerar IDs ativos, usar /progress (que revela correct) e submeter vários desafios para acumular XP no mesmo dia.

Impacto: Bypass da regra diária, inflação de XP e distorção de ranking/conquistas.

Condição: Semântica diária ativa, conforme README e chave Redis com TTL diário.

Correção recomendada: Vincular questions/progress/submit ao daily do chamador ou a um attempt token diário.

Evidência de código:

[backend/src/challenge/challenge.service.ts:52-56,85-87](#)

```
const challenge = await query.orderBy('RAND()').findOne();
await this.setRedisDailyChallenge(usuario_id, challenge);
const cacheKey = `daily-challenge:${usuario_id}`;
await this.redisService.set(cacheKey, JSON.stringify(challenge), ttl);
```

[backend/src/challenge/challenge.service.ts:213-223](#)

```
const challenge = await this.challengeRepository.findOne({
  where: { id: challengeId, active: true },
});
const existing = await this.usuarioChallengeRepository.findOne({
  where: { usuario_id, challenge_id: challengeId },
});
```

SEC-05 - Administrador de empresa controla catálogos e mídia globais

ALTA

Categoria: Banco sem tranca

Por que é explorável: Um administrador da empresa A chama os CRUDs por ID e altera, exclui ou sobrescreve treinamento consumido por todas as empresas.

Impacto: Comprometimento de integridade multi-tenant e possível indisponibilidade global de conteúdo.

Condição: Classificação pressupõe ADMIN delegado a clientes, como indica a UI/rotas de empresa. Se todo ADMIN for staff global, o risco passa a ser least privilege e papel ambíguo.

Correção recomendada: Criar PLATFORM_ADMIN para catálogo global ou adicionar empresa_id, filtros compostos e prefixo S3.

Evidência de código:

[backend/src/conteudo/modulo/modulo.controller.ts:31-49](#)

```
@Post()
@Roles(Role.ADMIN)
async create(@Body() dto: CreateModuloDto) { ... }
@Patch('/:id')
@Roles(Role.ADMIN)
...
@Delete('/:id')
@Roles(Role.ADMIN)
```

[backend/src/conteudo/s3/s3.service.ts:59-72](#)

```
return `modulos/${moduloId}/thumbnail`;
return `modulos/${moduloId}/aulas/${aulaId}/video`;
return `modulos/${moduloId}/aulas/${aulaId}/pages/${pageOrder}`;
```

SEC-06 - Endpoint do editor Vite sobrescreve fonte sem autenticação**ALTA**

Categoria: Permissão no navegador

Por que é explorável: Qualquer cliente que alcance a porta 5173 envia um biomeld e pontos e altera o repositório.

Impacto: Adulteração persistente de código/dados do desenvolvedor e HMR não confiável.

Condição: Somente vite serve; o plugin usa apply: 'serve'. Requer porta de desenvolvimento acessível.

Correção recomendada: Remover a escrita do Vite compartilhado ou limitar a loopback, flag local e endpoint autenticado.

Evidência de código:

[frontend/vite.config.ts:5-7.13-15](#)

```
plugins: [react(), tailwindcss(), devSaveRegionPlugin()],
server: { host: true, port: 5173, ... }
```

[frontend/src/prototvnes/worldman/devSaveRegionPlugin.ts:10-12.32-39](#)

```
apply: 'serve',
server.middlewares.use('/__save-region', async (req, res) => {
const source = await readFile(mockPath, 'utf8');
const updated = upsertRegion(source, biomeId, points);
await writeFile(mockPath, updated, 'utf8');
});
```

SEC-07 - Coordenadas viram JavaScript persistente em mockData.ts**ALTA**

Categoria: XSS

Por que é explorável: Uma string em y fecha o objeto, injeta um IIFE e reabre o objeto. Ao importar ou atualizar via HMR, o código roda no origin do frontend e pode agir com a sessão da vítima.

Impacto: XSS persistente no ambiente dev e alteração de código-fonte com ações autenticadas pela vítima.

Condição: Vite serve acessível e vítima carrega o módulo do mapa (ou recebe HMR).

Correção recomendada: Validar números finitos/faixa e persistir JSON com JSON.stringify, nunca TypeScript interpolado.

Evidência de código:

[frontend/src/prototvnes/worldman/devSaveRegionPlugin.ts:19-31](#)

```
const { biomeId, points } = JSON.parse(body) as { ... };
if (!biomeId || !Array.isArray(points)) { ... }
```

[frontend/src/prototvnes/worldman/devSaveRegionPlugin.ts:87-94](#)

```
const inner = points
.map((p) => ` ${indent} { x: ${p.x}, y: ${p.y} },`)
.join('\n');
return `region: [\n${inner}\n${indent}],`;
```

SEC-08 - Startup cria dez contas com a senha pública seed123**ALTA**

Categoria: Chaves expostas

Por que é explorável: Entrar no endpoint público de login com um dos e-mails seed e a senha publicada no código.

Impacto: Acesso não autorizado às funções autenticadas, manipulação de progresso, XP e recursos.

Condição: seed-1 não existia no primeiro startup; uma vez criadas, as contas persistem.

Correção recomendada: Remover seed do boot normal, bloquear em produção e eliminar/rotacionar contas existentes.

Evidência de código:

[backend/src/seed/seed.service.ts:22-24.49-58](#)

```
async onModuleInit() {
await this.seedRankingUsers();
}
```



```
const passwordHash = bcrypt.hashSync('seed123', 10);
for (const seed of seedUsers) {
  ... password: passwordHash, role: Role.USER ...
}
```

SEC-09 - Segredo JWT permanece em referência do histórico Git

ALTA

Categoria: Chaves expostas

Por que é explorável: Se algum ambiente ainda aceitar o valor antigo, um atacante que lê o Git pode assinar JWT com sub e role arbitrários, inclusive admin.

Impacto: Forja de sessão e possível comprometimento total do controle de acesso.

Condição: Algum deployment ainda usa o segredo histórico. O JWT local atual foi verificado como diferente.

Correção recomendada: Rotacionar todos os ambientes, invalidar sessões e reescrever todas as refs do histórico.

Evidência de código:

[commit 06bbb0c7](#) / [backend/src/auth/constants.ts:2-4](#)

```
export const jwtConstants = {
  secret: '<REDIGIDO: segredo real de 32 caracteres>',
};
```

[commit 06bbb0c7](#) / [backend/src/auth/auth.module.ts:12-15](#)

```
JwtModule.register({
  global: true,
  secret: jwtConstants.secret,
  signOptions: { expiresIn: '60s' },
}),
```

06 Recomendações priorizadas

Prioridade	Ação	Resultado esperado	Achado
P1	Isolar o editor dev	Desligar o endpoint por padrão, bind loopback, autorização, validação runtime e persistência JSON.	SEC-06/07
P2	Autenticar Socket.IO	Validar JWT/blacklist no handshake e derivar o room do sub.	SEC-01
P3	Eliminar credenciais seed	Remover SeedModule do boot e bloquear/rotacionar contas existentes.	SEC-08
P4	Rotacionar e limpar JWT	Invaldar sessões e reescrever todas as refs que preservam o segredo.	SEC-09
P5	Tenant-aware notifications	Comparar empresa do solicitante e do alvo em criar/listar/marcar todas.	SEC-02
P6	Separar papéis de plataforma	Criar PLATFORM_ADMIN ou tornar todos os catálogos tenant-scoped.	SEC-05
P7	Impor desbloqueio no GET	Verificar pré-requisito antes de retornar URL ou quiz de aula.	SEC-03
P8	Impor elegibilidade diária	Vincular questions/progress/submit ao daily ou attempt token diário.	SEC-04

Sequenciamento: P1-P4 reduzem superfícies de acesso imediato/credenciais; P5-P6 corrigem fronteiras de tenant e privilégio; P7-P8 fecham regras de progressão e integridade da economia.

07 ISSUES PARA O GITHUB

Textos completos em Markdown, prontos para copiar e colar. Achados SEC-06 e SEC-07 foram agrupados em uma única issue porque compartilham o mesmo endpoint e correção. Nenhuma issue foi publicada.

```
--- ISSUE 1 ---
# [Segurança] Autenticar assinaturas WebSocket por JWT

Labels sugeridas: `security`, `alta`

## Descrição do problema
O gateway usa `handshake.query.userId` como identidade e entra diretamente em `user_{id}`. Não há validação de assinatura/expiração JWT, cookie, blacklist Redis nem comparação com o usuário informado.

Isso é explorável porque qualquer cliente Socket.IO que alcance o gateway escolhe o ID de outra pessoa. CORS restringe navegadores, mas não autentica clientes nativos.

## Evidência
`backend/src/gateway/app.gateway.ts:24-35`
```ts
const userId = Number(client.handshake.query.userId);
if (!userId || isNaN(userId)) { client.disconnect(); return; }
client.join(`user_${userId}`);
```

`backend/src/gateway/app.gateway.ts:45-56`
```ts
const room = `user_${payload.usuario_id}`;
this.server.to(room).emit('new-notification', payload);
```

## Impacto
Leitura de notificações futuras de outra pessoa, inclusive entre empresas.

## Sugestão de correção
Validar JWT no handshake usando o mesmo segredo, expiração e blacklist do HTTP. Derivar a sala exclusivamente de `payload.sub` e ignorar `query.userId`.

## Critérios de aceite
- [ ] Conexão sem token válido é recusada.
- [ ] Token expirado ou em blacklist é recusado.
- [ ] O room é derivado do `sub` autenticado.
- [ ] Informar outro `userId` não muda o room.
- [ ] Teste e2e prova que usuário A não recebe notificação de B.
--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---
# [Segurança] Restringir notificações ao tenant do administrador

Labels sugeridas: `security`, `alta`

## Descrição do problema
`OwnershipGuard` libera todo `Role.ADMIN` antes da checagem de dono. As rotas de criar, buscar e marcar todas aceitam `usuario_id` controlado pelo cliente, e o service não valida `empresa_id`.

Um admin da empresa A pode agir sobre notificações de um usuário da empresa B.

## Evidência
`backend/src/auth/ownership.guard.ts:23-25`
```ts
if (user.role === Role.ADMIN) return true;
```

`backend/src/notification/notification.controller.ts:26-48,70-76`
```ts
@OwnerField('usuario_id', 'body')
@OwnerField('id', 'query')
@OwnerField('usuario_id', 'params')
```

`backend/src/notification/notification.service.ts:99-135`
```ts
where: { usuario_id }
```

## Impacto
Quebra cross-tenant de confidencialidade e integridade: leitura, inserção e alteração de notificações.

## Sugestão de correção
Passar o solicitante ao service e comparar a empresa do alvo com a empresa do admin. Se admin não precisa operar notificações de
```

terceiros, remover o bypass.

```
## Critérios de aceite
- [ ] Admin só acessa usuários da própria empresa.
- [ ] ID de outra empresa retorna 403 ou 404.
- [ ] Usuário comum continua restrito ao próprio ID.
- [ ] Testes cobrem criar, buscar e marcar todas entre duas empresas.
--- FIM ISSUE 2 ---
```

--- ISSUE 3 ---

[Segurança] Impor desbloqueio antes de retornar conteúdo da aula

Labels sugeridas: `security`, `média`

Descrição do problema

O frontend desabilita aulas com status `locked`, mas `GET /conteudo/aulas/:id` busca apenas `{ id, active: true }` e retorna vídeo, páginas com URL S3 assinada e quiz sem executar `verificarDesbloqueio`.

Evidência

```
`frontend/src/components/sections/HomePage/Conteudos/AulaListItem.tsx:19-26`
```tsx
```

```
const isLocked = aula.status === 'locked';
<button disabled={isLocked} />
```
```

```
`backend/src/conteudo/aula/aula.service.ts:49-100`
```

```
```ts
where: { id, active: true }
// retorna content_url, pages e quiz sem verificarDesbloqueio
```
```

`backend/src/conteudo/aula/aula.service.ts:354-377` contém a validação já usada nas escritas.

Impacto

Usuário autenticado acessa material e perguntas antes de concluir a etapa anterior.

Sugestão de correção

Chamar `verificarDesbloqueio(aula, usuario\_id)` antes de resolver/assinar URLs. Para aula bloqueada, devolver 403 ou apenas metadados mínimos.

Critérios de aceite

```
- [ ] GET de aula locked retorna 403 sem URLs ou quiz.
- [ ] Após concluir a anterior, o mesmo GET retorna conteúdo.
- [ ] Não é gerada URL S3 antes da autorização.
- [ ] Teste e2e cobre acesso direto por ID.
--- FIM ISSUE 3 ---
```

--- ISSUE 4 ---

[Segurança] Vincular progresso e XP ao desafio diário

Labels sugeridas: `security`, `média`

Descrição do problema

O backend seleciona/cacheia um desafio por usuário/dia, mas `questions`, `progress` e `submit` aceitam qualquer `challengeId` ativo. O ID recebido nunca é comparado ao valor `daily`.

Evidência

```
`backend/src/challenge/challenge.service.ts:36-58,73-87`
```ts
```

```
const cacheKey = `daily-challenge:${usuario_id}`;
await this.redisService.set(cacheKey, JSON.stringify(challenge), ttl);
```
```

`backend/src/challenge/challenge.controller.ts:29-56` e `challenge.service.ts:113-230` operam sobre ID arbitrário.

`challenge.service.ts:265-295` concede XP.

Impacto

Conclusão de vários desafios no mesmo dia, inflação de XP e distorção de ranking/conquistas.

Sugestão de correção

Resolver o desafio diário do chamador em todas as três operações e exigir igualdade, ou emitir um attempt token diário vinculado a usuário, desafio e data.

Critérios de aceite

```
- [ ] Apenas o desafio diário pode fornecer perguntas/progresso/submit.
- [ ] ID ativo diferente retorna 403.
- [ ] Não há concessão de XP para tentativa inelegível.
- [ ] Teste e2e cobre dois challenges ativos no mesmo dia.
--- FIM ISSUE 4 ---
```

--- ISSUE 5 ---

[Segurança] Separar administração de empresa e catálogo global

Labels sugeridas: `security`, `alta`

Descrição do problema

O único papel `ADMIN` é usado para administrar uma empresa e também para mutar módulos, aulas, quizzes, Benefits e mídia globais. Esses objetos não têm `empresa\_id`; updates/deletes usam apenas ID e as chaves S3 não incluem tenant.

Condição: o risco cross-tenant se materializa quando `ADMIN` é delegado a clientes, como indica a UI e as rotas `/admin/empresa`. Se todos os admins forem operadores globais, ainda falta um papel explícito e least privilege.

Evidência

```
`backend/src/auth/roles.enum.ts:1-4`, `admin.controller.ts:11-12`
```

```
`backend/src/conteudo/modulo/modulo.controller.ts:31-49`, `aula.controller.ts:32-50`, `aula-quiz.controller.ts:24-45`,  
`conteudo/s3/upload.controller.ts:31-44`, `benefits/benefits.controller.ts:21-24,35-38`
```

```
`backend/src/conteudo/s3/s3.service.ts:59-72`  
````ts  
return `modulos/${moduloId}/aulas/${aulaId}/video`;
````
```

Impacto

Admin de uma empresa pode alterar/excluir/reescrever conteúdo visto por todas as empresas.

Sugestão de correção

Criar `PLATFORM\_ADMIN` exclusivo para catálogos globais. Se o conteúdo for por empresa, adicionar `empresa\_id`, filtros compostos e prefixo S3 por tenant em todos os reads/writes.

Critérios de aceite

- [] ADMIN de empresa não muta catálogo global.
- [] PLATFORM_ADMIN é explicitamente provisionado e auditável.
- [] IDs de outro tenant retornam 403/404 quando o catálogo for tenant-scoped.
- [] Chaves S3 incluem tenant ou são restritas ao papel de plataforma.
- [] Testes usam duas empresas e dois administradores.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Segurança] Isolar o editor dev e bloquear injeção em mockData.ts

Labels sugeridas: `security`, `alta`

Descrição do problema

O plugin do Vite expõe `POST /\_\_save-region` sem sessão/papel, com `host: true`, e sobrescreve `mockData.ts`. Além disso, o cast TypeScript não valida o JSON em runtime e `p.x`/`p.y` são interpolados diretamente em código TypeScript.

Uma string manipulada fecha o objeto, injeta um IIFE e persiste JavaScript executado ao importar/HMR do mapa.

Evidência

```
`frontend/vite.config.ts:5-7,13-15`  
````ts  
plugins: [react(), tailwindcss(), devSaveRegionPlugin()],
server: { host: true, port: 5173 }
````
```

```
`frontend/src/prototypes/worldmap/devSaveRegionPlugin.ts:19-39,87-94`  
````ts  
if (!biomeId || !Array.isArray(points)) { ... }
.map((p) => `${indent} { x: ${p.x}, y: ${p.y} },`)
await writeFile(mockPath, updated, 'utf8');
````
```

Impacto

Adulteração remota do repositório e XSS persistente no origin do frontend de desenvolvimento.

Sugestão de correção

Remover o endpoint de ambientes compartilhados; exigir flag local e bind loopback; autenticar/autorizar; validar schema runtime (`number`, finito, faixa 0..100, limites); persistir JSON via `JSON.stringify` em vez de gerar `.ts`.

Critérios de aceite

- [] Endpoint não existe fora de modo local explícito.
- [] Sem sessão/papel autorizado retorna 401/403.
- [] Strings, NaN, infinito e fora de faixa retornam 400 sem alterar arquivo.
- [] Payload de injeção não compila nem executa.
- [] Testes confirmam persistência segura em dados, não código.

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

[Segurança] Remover contas seed com credencial pública

Labels sugeridas: `security`, `alta`

Descrição do problema

`SeedModule` é carregado no boot normal e cria dez contas previsíveis com a mesma senha hardcoded `seed123`. A checagem evita nova inserção, mas não remove nem desativa contas já criadas.

Evidência

```
`backend/src/app.module.ts:12,52`
```

```
`backend/src/seed/seed.service.ts:22-24,26-40,42-58`
```

```
````ts
async onModuleInit() { await this.seedRankingUsers(); }
const passwordHash = bcrypt.hashSync('seed123', 10);
````
```

`backend/src/auth/auth.controller.ts:26-34` expõe o login público.

Impacto

Acesso não autorizado às áreas autenticadas e manipulação de progresso/XP.

Sugestão de correção

Retirar a seed do startup. Permitir dados demo apenas com flag explícita fora de produção. Excluir, bloquear ou rotacionar as dez contas existentes.

Critérios de aceite

- [] Boot de produção não cria `seed-%@secureplay.dev`.
 - [] Aplicação falha se flag demo estiver ativa em produção.
 - [] Login com `seed123` falha para todas as contas seed.
 - [] Migração/operacional remove ou bloqueia contas existentes.
 - [] Teste de integração cobre o boot de produção.
- FIM ISSUE 7 ---

--- ISSUE 8 ---

[Segurança] Rotacionar e remover segredo JWT do histórico Git

Labels sugeridas: `security`, `alta`

Descrição do problema

O commit `06bbb0c7f79939cded30218cf82d92571be6e9e2` contém um segredo JWT real de 32 caracteres em `backend/src/auth/constants.ts:3` e o usa no `JwtModule`. A remoção no commit `3641d626...` não limpou o objeto; `refs/remotes/origin/master` ainda aponta para o commit vulnerável.

O valor foi redigido neste relatório. O segredo local atual é diferente, mas isso não prova a rotação em todos os ambientes.

Evidência

```
`commit 06bbb0c7... / backend/src/auth/constants.ts:2-4`
````ts
```

```
export const jwtConstants = {
 secret: '<REDIGIDO: segredo real de 32 caracteres>',
};
````
```

```
`commit 06bbb0c7... / backend/src/auth/auth.module.ts:12-15`
```

```
````ts
secret: jwtConstants.secret,
````
```

Impacto

Se ainda aceito por qualquer ambiente, permite forjar JWT com usuário e papel arbitrários.

Sugestão de correção

Rotacionar todos os ambientes, invalidar sessões, reescrever todas as refs e coordenar force-push/novos clones. Manter segredo somente em secret manager e adicionar scanner no CI/pre-commit.

Critérios de aceite

- [] Token assinado com o segredo antigo é rejeitado em todos os ambientes.
 - [] Todas as sessões antigas foram invalidadas.
 - [] Busca em todas as refs não encontra o literal.
 - [] `origin/master` não aponta para objeto vulnerável.
 - [] Startup rejeita segredo ausente ou fraco.
 - [] Scanner de segredos bloqueia regressões.
- FIM ISSUE 8 ---

Fim do relatório. Contagens e evidências refletem o worktree local auditado em 30/08/2026; alterações posteriores exigem nova execução do script e nova revisão de código.